

Laboratorio 4 EL5859 - Transcripción de la conversación

2026-09-18 · @Someone

Contexto

Esta sesión cubrió los incisos A, B y C del Laboratorio Semana 5 de EL5859 Computación Heterogénea (TEC, prof. Luis G. León-Vega): profiling de una aplicación de colimación de nubes de puntos, con dos perfiles en forma de H de 100 000 puntos cada uno dentro de una ventana de 10000 x 10000.

Se usaron `perf`, Google Performance Tools y Valgrind (Callgrind) en un Dell G16 7620 con CPU híbrida; el programa corrió siempre en un núcleo de eficiencia (`cpu_atom`). Los incisos D (instrumentación manual) y E (optimización) quedaron pendientes.

Los mensajes están en orden cronológico. Las respuestas aparecen resumidas y los logs de terminal, que se repetían cinco veces, están recortados a lo relevante.

Inciso A: centroides y ejecución base

¿Qué representan los centroides de ambos perfiles?

Usuario: adjuntó el PDF de la guía y preguntó qué representan los centroides.

Claude: son el centro de masa de cada perfil, es decir, el promedio de las coordenadas x , y de todos sus puntos. El centroide del perfil objetivo queda fijo; el del perfil fuente se recalcula en cada iteración porque el perfil se mueve. Al estimar la transformación rígida (estilo ICP) se restan los centroides, la rotación se calcula alrededor de ese centro y la traslación queda como su diferencia. La distancia entre centroides sirve como medida simple de posición global, pero no mide orientación ni forma.

¿Se ve como una constelación de comunicaciones eléctricas?

Usuario: preguntó si la nube de puntos se ve como una constelación de comunicaciones eléctricas.

Claude: la analogía funciona en parte. La nube de puntos en el plano x , y se parece a una constelación en el plano I/Q : la rotación equivale a un desfase de portadora, la traslación a un offset DC, y la deformación y el ruido a ruido del canal. En comunicaciones el centroide ideal es el origen, y aquí la distancia entre centroides mide cuánto está desplazada una

nube respecto a la otra. La diferencia es que los símbolos de una constelación son discretos y conocidos, mientras que aquí la correspondencia entre puntos no se conoce y hay que buscarla con el vecino más cercano.

Revisión de la ejecución base

Usuario: pegó la salida de `./point_cloud_collimation` (45 iteraciones) y pidió revisarla.

Claude: la transformación esperada es coherente con la sintética, el centroide del objetivo es constante y el ángulo se recupera muy bien, pero la traslación queda mal recuperada.

Métrica	Inicial	Final
Distancia entre centroides	760.12	600.87
<code>chamfer_rmse</code>	470.75	111.00
Cobertura	80.11 %	96.94 %
Ángulo (esperado -18.00000)	0	-18.00595
tx (esperado -1757.15)	0	-1720.41
ty (esperado 2390.35)	0	1781.98

Claude: el error de ty es de unas 608 unidades, mucho mayor que la deformación (rms 56.8), así que la hipótesis fue un mínimo local del ICP, con los perfiles solapados pero desplazados a lo largo de sus rieles. También se verificó que `chamfer_rmse` es la raíz del promedio de los cuadrados de `rmse_s2t` y `rmse_t2s`, y que `max=14142.1` (la diagonal del canvas) corresponde a puntos sin vecino hasta la iteración 3.

Aclaración y ejecución con `--export`

Usuario: aclaró que la consulta era del inciso B y luego corrió

`./point_cloud_collimation --export`.

Claude: la salida fue idéntica (mismas 45 iteraciones y mismo `profile_score` final de 0.01847086), así que el programa es determinista y el cálculo es el mismo con y sin exportación.

Usuario: preguntó si con `--export` no debería tardar más.

Claude: sí, porque escribe CSV y PPM por cada paso, pero el log no imprime tiempos; se midió después con `perf stat`.

Inciso B: perf record y perf report

Primer perf record

Usuario: corrió `perf record -g ./point_cloud_collimation` (antes de `perf stat`).

Claude: los avisos sobre `kptr_restrict` y `perf_event_paranoid` solo afectan a las muestras del kernel; los símbolos del programa se resuelven normal. Capturó 138 949 muestras (10.736 MB), unos 35 s de CPU con la frecuencia por defecto de 4000 Hz (estimación, no tiempo medido).

perf report

Usuario: sacó `perf report` a un `.txt` de 4.1 MB, con el árbol de llamadas incluido.

Claude: el evento fue `cpu_atom/cycles/P` (137 K muestras, 119.57 G ciclos aprox.), es decir, el programa corrió en un núcleo de eficiencia. Casi todo el tiempo está en una sola función.

Función	Self	Children
<code>GridIndex::nearest</code>	97.14 %	97.67 %
<code>GridIndex::GridIndex</code> (constructor)	0.65 %	0.73 %
<code>percentile</code>	0.51 %	0.51 %
<code>compare_profiles</code>	0.18 %	0.19 %
<code>estimate_rigid_transform</code> (inlined)	0.00 %	0.05 %
<code>add_random_deformation</code> (inlined)	0.00 %	0.04 %

Claude: dentro de `nearest`, la búsqueda en la tabla hash (`unordered_map::find`, integrada) acumula 23.30 %: `_M_find_before_node` 19.2 % y `_M_bucket_index` 3.7 %. `vector::operator[]` suma 4.34 %.

Claude: también aparecieron pilas basura (`[unknown]`), direcciones tipo `0x4092971...` porque el binario se compiló sin `-fno-omit-frame-pointer`; por eso no se podía saber si `nearest` era llamada desde el paso del ICP o desde `compare_profiles`.

Inciso B: perf stat con y sin --export

Qué mide perf stat

Usuario: preguntó si se podían analizar las "muestras" de `perf stat`.

Claude: `perf stat` no genera muestras sino contadores totales de toda la ejecución. Con ellos se calculan las instrucciones por ciclo (IPC), la tasa de fallos de branch y, si hubiera eventos de caché, la tasa de fallos de caché. Propuse añadir `-e` con los eventos de caché y redirigir la salida a `/dev/null`.

Usuario: prefirió apegarse a los comandos exactos de la guía, porque las comparativas se hacen con sus compañeros con los resultados de la misma.

Resultados con el comando de la guía

Usuario: corrió primero `perf stat ./point_cloud_collimation --export` y después `perf stat ./point_cloud_collimation`.

Métrica	Sin --export	Con --export	Diferencia
Tiempo total	33.52 s	36.60 s	+3.08 s (+9.2 %)
User / sys	33.44 / 0.079 s	36.35 / 0.166 s	+2.92 s / +0.09 s
Ciclos	116.5 G	126.9 G	+8.9 %
Instrucciones	206.1 G	238.3 G	+15.6 %
IPC	1.8	1.9	
Branches	27.4 G	33.7 G	+23 %
Branch misses	537 M (2.0 %)	558 M (1.7 %)	+3.8 %
Topdown: retiring / backend / frontend / bad speculation	35.9 / 30.5 / 16.8 / 16.8 %	38.4 / 28.7 / 17.1 / 15.9 %	
Cambios de contexto / page faults	347 / 43 979	418 / 37 298	
cache-references y cache-misses	no disponibles	no disponibles	

Claude: exportar cuesta unos 3 s (9 %), casi todo CPU en espacio de usuario (generar el texto de los CSV y las imágenes) y solo 0.09 s más de kernel. Ese código es muy predecible, por eso sube el IPC y baja la tasa de fallos de branch. Los page faults salieron más bajos con exportación, lo cual no se atribuyó a la exportación. La corrida sin exportar es la más representativa del algoritmo y está más limitada por backend (30.5 %), coherente con accesos indirectos en `nearest`.

Limitaciones a anotar en el reporte

Claude: estas limitaciones conviene anotarlas en el reporte.

- Los eventos `cpu_core/...` salieron como `<not counted>`: la CPU es híbrida y el programa corrió siempre en un núcleo de eficiencia (`cpu_atom` , 0 migraciones), por lo que los contadores del núcleo de rendimiento nunca se activaron.
- Los porcentajes entre paréntesis (50 %, 60 %) indican multiplexado: cada contador estuvo activo solo parte del tiempo y los totales son estimaciones escaladas; las razones (IPC, tasa de fallos) son más confiables.
- `cache-references` y `cache-misses` no aparecen en la salida por defecto y hay que pedirlos con `-e`.
- Cada configuración se corrió una sola vez.

Inciso B: Callgrind y Google Performance Tools

Callgrind

Usuario: corrió `valgrind --tool=callgrind ./point_cloud_collimation` y leyó el resultado con `callgrind_annotate callgrind.out.28716 | head -40`; después guardó el reporte completo en `callgrind_annotate.txt`.

Claude: el total fue de 205 733 272 629 instrucciones (`Ir`), casi igual a las 206.1 G de `perf stat` sin exportar (0.2 % de diferencia). Callgrind separa por archivo el código de cabeceras integrado en `GridIndex::nearest`, por lo que todas estas líneas son la misma función:

Origen (dentro de <code>GridIndex::nearest</code>)	Ir	% del total
Código propio (<code>point_cloud_collimation.cpp</code>)	166 674 287 838	81.01
<code>stl_vector.h</code>	20 804 515 668	10.11
<code>hashtable.h</code>	7 290 929 053	3.54
<code>stl_algobase.h</code>	3 680 330 298	1.79
<code>hashtable_policy.h</code>	3 484 986 660	1.69
<code>stl_function.h</code>	617 513 896	0.30
<code>stl_iterator.h</code>	214 381 544	0.10
Suma de <code>nearest</code>	202 766 944 957	98.56

Claude: el resto es despreciable: constructor de `GridIndex` 0.21 %, `sincos` 0.12 %, `collimate_icp` 0.11 % y `compare_profiles` 0.10 %. Como limitación, Callgrind por defecto solo registra `Ir`, sin simulación de caché ni de branches.

Google Performance Tools

Usuario: recompiló con `libprofiler` (`make clean && make CXXFLAGS=... GST_LIBS=...`), verificó el enlace con `ldd ./point_cloud_collimation | grep profiler`, corrió `CPUPROFILE=point_cloud.prof ./point_cloud_collimation` y leyó el perfil con `google-pprof -text`. Obtuvo 3489 muestras (34.89 s) y guardó el reporte en `gperftools_report.txt`.

Función	flat	flat %	cum	cum %
<code>GridIndex::nearest</code>	25.09 s	71.91	34.17 s	97.94
<code>_M_find_before_node</code>	5.69 s	16.31	7.08 s	20.29
<code>_Mod_range_hashing::operator()</code>	1.39 s	3.98	1.39 s	3.98
<code>equal_to::operator()</code>	1.35 s	3.87	1.35 s	3.87
<code>vector::operator[]</code>	0.35 s	1.00	0.35 s	1.00
<code>_M_locate</code>	0.05 s	0.14	8.32 s	23.85
<code>_M_bucket_index</code>	0.04 s	0.11	1.43 s	4.10
<code>compare_profiles</code>	0.01 s	0.03	23.29 s	66.75
<code>nearest_neighbor_distances</code>	0.01 s	0.03	22.76 s	65.23
<code>collimate_icp</code>	0.03 s	0.09	34.86 s	99.91

Claude: el hallazgo principal es que dos tercios del tiempo (66.75 % acumulado) se gastan en calcular las métricas de comparación (`compare_profiles` → `nearest_neighbor_distances` → `nearest`), no en el emparejamiento del ICP. Por diferencia, los 34.17 s de `nearest` menos 22.76 s dan unos 11.4 s (~33 %) para las búsquedas del propio `collimate_icp`. Eso cuadra con dos búsquedas por iteración para las métricas (s2t y t2s) frente a una del ICP; es una inferencia que se confirmaría con la instrumentación del inciso D. Además explica por qué `compare_profiles` parecía irrelevante en `perf` (0.18 % propio) y en Callgrind (0.10 %): su costo está en la función que llama. Este binario lleva `-fno-omit-frame-pointer`, a diferencia del usado en `perf` y Callgrind.

Comparación entre perf, gperftools y Valgrind

Usuario: ¿Los resultados coinciden entre perf, Google Performance Tools y Valgrind?
¿Puedes explicar las diferencias?

Claude: Coinciden en lo importante: las tres herramientas señalan a `GridIndex::nearest` como el hotspot. Los porcentajes difieren porque cada una mide algo distinto y agrupa el código integrado (inlined) de otra forma.

Herramienta	Qué mide	GridIndex::nearest (propio)	GridIndex::nearest (con lo que integra)
perf	Muestreo de ciclos de CPU	97,14 % Self	97,67 % Children
Google Performance Tools	Muestreo a 100 Hz (3489 muestras)	71,91 % <code>flat</code>	97,94 % <code>cum</code>
Callgrind	Instrucciones ejecutadas (<code>Ir</code>), sin muestreo	81,01 %	98,56 %

Claude: las diferencias vienen de tres causas. Primero, perf y gperftools son estadísticos (muestreo) y Callgrind cuenta exactamente cada instrucción. Segundo, gperftools y Callgrind separan funciones auxiliares de la tabla hash (`_M_locate` , `_M_find_before_node` , `stl_vector.h`), que perf atribuye directamente a `nearest` porque están integradas en su código. Tercero, contar instrucciones no es medir tiempo: una instrucción que espera memoria pesa más en perf que en Callgrind.

Claude: como comprobación cruzada, el total de instrucciones de Callgrind (205 733 272 629) y el de `perf stat` difieren aproximadamente 0,2 %, lo que indica que las mediciones son consistentes entre sí.

Borrador del README, sección 2.1 (hotspots)

Usuario: Ayúdame a generar esta sección para un README con el formato de mi compañera, para que sea simétrico y estético. Puedes poner valores; la explicación la lleno yo. Pon `Milagro` como referencia de la carpeta.

Claude: este es el borrador con los valores de las tres herramientas. Los `[COMPLETAR]` son las ejecuciones con `--export` que faltan por correr.

Los archivos completos obtenidos durante el profiling se encuentran en la carpeta `Milagro/`.

2.1 Hotspots identificados

perf

En la ejecución normal, `perf report` identificó a `GridIndex::nearest()` como el principal hotspot del programa, con aproximadamente un 97.14 % Self (97.67 % acumulado).

En la ejecución con `--export`, [COMPLETAR: correr `perf record -g ./point\_cloud\_collimation --export` y reportar el % Self de `GridIndex::nearest()`].

Google Performance Tools

En la ejecución normal se obtuvieron 3489 muestras (34.89 s). La función `GridIndex::nearest()` representó:

- * 71.91 % de las muestras directas (`flat`).
- * 97.94 % de las muestras acumuladas (`cum`).

El árbol de llamadas muestra que una parte importante del tiempo ocurre mediante:

```
collimate_icp()                99.91 % cum
└─ compare_profiles()          66.75 % cum
   └─ nearest_neighbor_distances() 65.23 % cum
      └─ GridIndex::nearest()
```

También aparecen funciones relacionadas con las estructuras utilizadas por `GridIndex::nearest()`:

- * `std::\_Hashtable::\_M\_locate`: 23.85 % `cum`.
- * `std::\_Hashtable::\_M\_find\_before\_node`: 16.31 % `flat`, 20.29 % `cum`.
- * `std::\_\_detail::\_Mod\_range\_hashing::operator()`: 3.98 % `flat`.
- * `std::equal\_to::operator()`: 3.87 % `flat`.
- * `std::vector::operator[]`: 1.00 % `flat`.

Con `--export`, [COMPLETAR: correr `CPUPROFILE=... ./point\_cloud\_collimation --export` y reportar el `flat` y `cum` de `GridIndex::nearest()`].

Valgrind / Callgrind

Callgrind contabilizó en la ejecución normal:

```
* 205,733,272,629 instrucciones (`Ir`) totales.
* 166,674,287,838 instrucciones directamente asociadas al código de
`GridIndex::nearest()`.
* Esto corresponde aproximadamente al 81.01 % del total.
```

También se observa trabajo asociado a `std::vector` y a la tabla hash:

```
* `stl_vector.h`: 20,804,515,668 instrucciones (10.11 %).
* `hashtable.h`: 7,290,929,053 instrucciones (3.54 %).
* `hashtable_policy.h`: 3,484,986,660 instrucciones (1.69 %).
```

Sumando todo el código integrado (inlined) dentro de `GridIndex::nearest()`, la función acumula 202,766,944,957 instrucciones, aproximadamente el 98.56 % del total.

El archivo también quedó guardado como `README_seccion_2.1_hotspots.md`.

Inciso C: ensamblador, objdump y perf annotate

Usuario: Para el C, ayúdame a montar también el README. Revisa primero lo que tengo en la terminal. Las funciones que pide la guía estaban dentro de `-O2` y por eso no me aparecían, pero mis compañeros me ayudaron y ya saqué el `.txt`. Las explicaciones déjamelas llenar a mí.

Claude: revisé la salida de la terminal y el archivo `perf_annotate.txt` que subiste. Con `-O2` el compilador integra (inline) varias funciones, así que no aparecen como símbolos propios; solo `GridIndex::nearest`, `compare_profiles` y `render_motion_frame` quedaron como regiones identificables. Las otras dos se ubicaron por las llamadas a la librería matemática que contienen.

Comandos usados:

```
g++ -std=c++17 -O2 -g -S -masm=intel $(pkg-config --cflags gstreamer-1.0
gstreamer-app-1.0) point_cloud_collimation.cpp -o point_cloud_collimation.s
objdump -drwC -Mintel ./point_cloud_collimation >
point_cloud_collimation.objdump
perf record -g ./point_cloud_collimation
perf annotate [comando exacto pendiente] > perf_annotate.txt
```

Región	Ubicación en el desensamblado
<code>GridIndex::nearest</code>	0xc9e0 (línea 8066)

Región	Ubicación en el desensamblado
<code>compare_profiles</code>	0x9910 (línea 5588); auxiliar <code>nearest_neighbor_distances</code> en 0x9630
<code>estimate_rigid_transform</code>	Integrada en <code>collimate_icp</code> (0xabd0); <code>atan2@plt</code> en 0xb302 y <code>sincos@plt</code> en 0xb31d
<code>add_random_deformation</code>	Integrada en <code>main</code> (0x52c0); <code>exp@plt</code> en 0x69e1 y <code>sqrt@plt</code> en 0x6ab5
<code>render_motion_frame</code>	0x8cc0 (línea 4800)

Claude: en `perf annotate` hubo 144 971 muestras para `cpu_atom/cycles/P`, de las cuales 140 993 están en `GridIndex::nearest`. Al principio asumí que las 144 971 eran solo de `nearest`; lo corregí a 140 993 al revisar el archivo.

Claude: una advertencia sobre un comando anterior: `perf annotate --stdio GridIndex::nearest > perf_annotate_nearest.txt` falló con "no samples" y dejó un archivo vacío. Mi búsqueda con `grep` sobre el desensamblado tampoco cubrió `sin@plt`, `cos@plt`, `hypot@plt` ni `lround@plt`, así que ese dato queda pendiente en el README.

Borrador del README, sección 3 (ensamblador)

Usuario: para el README usa esta misma estructura, con las tablas y demás (adjuntó el texto de la sección 3 de su compañera).

Claude: este es el borrador. Las explicaciones quedan como `[EXPLICACIÓN]` para que las llenes tú; los `[COMPLETAR]` son datos que solo tú puedes confirmar.

3.1. Evidencias

Archivos:

- * `[Ensamblador]`(Milagro/point_cloud_collimation.s)
- * `[Desensamblado]`(Milagro/point_cloud_collimation.objdump)
- * `[Anotación de perf]`(Milagro/perf_annotate.txt)

Comandos utilizados:

```
```bash
g++ -std=c++17 -O2 -g -S -masm=intel $(pkg-config --cflags gstreamer-1.0
gstreamer-app-1.0) point_cloud_collimation.cpp -o point_cloud_collimation.s
objdump -drwC -Mintel ./point_cloud_collimation >
point_cloud_collimation.objdump
perf record -g ./point_cloud_collimation
```

```
perf annotate [COMPLETAR: comando exacto usado] > perf_annotate.txt
...
```

La ejecución analizada con `perf annotate` contiene 144 971 muestras para el evento `cpu\_atom/cycles/P`, de las cuales 140 993 corresponden a `GridIndex::nearest`. Los porcentajes corresponden a `percent: local period`: indican cómo se distribuyen las muestras dentro de la región anotada, no el porcentaje total del programa.

### ### 3.2. Revisión de las cinco regiones

```
GridIndex::nearest
```

[EXPLICACIÓN]

Instrucciones observadas en el ensamblador:

Elemento	Instrucciones	Direcciones
Cálculo del índice de celda	`divsd`, `cvtsd2si` (con `cmplsd` y `subsd`)	0xca17, 0xca29 - 0xca5c, 0xca6a, 0xca7f - 0xcaae
Búsqueda del bucket en la tabla hash	`divq %r10`	0xcb59, 0xcb97
Recorrido de los nodos del bucket	`movq (%rax),%rcx`, `cmpq %rdi,%rsi`, `jne`	0xcb7f - 0xcba9
Acceso al candidato	`movslq (%rdx),%rax`, `shlq \$0x4,%rax`, `addq %rdi,%rax`	0xcbd0, 0xcddf, 0xcbe3
Cálculo de la distancia al cuadrado	`subsd`, `mulsd`, `addsd`	0xcbe6 - 0xcbf7
Comparación con la mejor distancia	`comisd`, `jbe`	0xcc00, 0xcc04
Control de flujo	`jbe`, `je`, `jne`, `cmovs`, `cmovl`	0xca27, 0xcb17, 0xcb24, 0xcb2a, 0xcb31, 0xcb4a, 0xcb6a

```
compare_profiles
```

[EXPLICACIÓN]

```
estimate_rigid_transform
```

[EXPLICACIÓN]

```
add_random_deformation
```

[EXPLICACIÓN]

```
render_motion_frame
```

## [EXPLICACIÓN]

## #### Ubicación de las regiones

Región	Ubicación observada
`GridIndex::nearest`	Símbolo principal en 0xc9e0 (línea 8066); tabla hash entre 0xcb45 y 0xcbab; candidatos desde 0xcbd0.
`compare_profiles`	Región principal en 0x9910 (línea 5588), `[clone.constprop.0]`; fragmento `[clone.cold]` en 0x3b32 (línea 821); `nearest_neighbor_distances` en 0x9630 (línea 5408).
`estimate_rigid_transform`	Con `-O2` no aparece como símbolo independiente. `atan2@plt` en 0xb302 (línea 6944) y `sincos@plt` en 0xb31d (línea 6948), dentro de `collimate_icp` (0xabd0, línea 6612).
`add_random_deformation`	Con `-O2` no aparece como símbolo independiente. `exp@plt` en 0x69e1 (línea 3002) y `sqrt@plt` en 0x6ab5 (línea 3043), dentro de `main` (0x52c0, línea 1953). [COMPLETAR: llamadas a `sin`, `cos` o `hypot`]
`render_motion_frame`	Región principal en 0x8cc0 (línea 4800), `[clone.constprop.0]`.

## ### 3.3. ¿Qué instrucciones concentran más muestras?

Dirección	Instrucción	% local	Operación
0xcbe6	`subsd (%rax),%xmm0`	16,67 %	Diferencia entre coordenada de la consulta y del candidato.
0xcc00	`comisd %xmm0,%xmm1`	7,48 %	Compara con la mejor distancia.
0xcbf7	`addsd %xmm1,%xmm0`	6,43 %	Suma los dos términos (distancia al cuadrado).
0xcbd0	`movslq (%rdx),%rax`	4,88 %	Obtiene el índice del candidato.
0xcc04	`jbe 0xcc19`	4,07 %	Salto según la comparación.
0xcb70	`movq (%rbx),%rax`	4,05 %	Lee un nodo del bucket.
0xcb76	`movq 0x8(%rax),%rdi`	4,01 %	Lee la clave del nodo.
0xcbeef	`mulsd %xmm0,%xmm0`	3,97 %	Cuadrado de la primera diferencia.
0xcb60	`movq (%rax,%rdx,8),%rbx`	3,26 %	Accede al bucket de la tabla hash.
0xcb97	`divq %r10`	3,16 %	División para calcular el bucket del siguiente nodo.

## [EXPLICACIÓN]

Región de la función	Direcciones	% local
Cálculo del índice de celda	0xca17 - 0xcab2	0,32 %

```
| Búsqueda en la tabla hash | 0xcb45 - 0xcbb0 | 23,33 % |
| Recorrido de candidatos y distancia | 0xcbd0 - 0xcc1d | 65,32 % |
| Resto de la función | - | 10,97 % |
```

La suma es 99,94 % por el redondeo de ``perf annotate``.

### 3.4. ¿Coinciden con el hotspot del ejercicio B?

En B, ``perf report`` mostró a ``GridIndex::nearest`` con 97,14 % Self y 97,67 % Children. Google Performance Tools la registró con 71,91 % ``flat`` y 97,94 % acumulado. Callgrind concentró 81,01 % de las instrucciones en esta función y 98,56 % al incluir el código integrado.

En ``perf annotate``, la búsqueda en la tabla hash concentra 23,33 % de las muestras locales, cercano al 23,3 % de ``unordered_map::find`` en ``perf report`` y al 23,85 % de ``_M_locate`` en Google Performance Tools.

[EXPLICACIÓN]

### 3.5. ¿Qué cambio intentaríamos primero?

[EXPLICACIÓN]

## Preguntas finales

### Qué instrucciones concentran más muestras

**Usuario:** ¿Qué instrucciones concentran más muestras? ¿Esto varía según la computadora o según la instrucción? Yo entendía que dependía de la instrucción.

**Claude:** depende de las dos cosas. Qué instrucciones son las costosas lo determina el código: en `GridIndex::nearest` siempre serán las que leen memoria ( `subsd (%rax),%xmm0, movslq (%rdx),%rax` ) y las de la tabla hash. Los porcentajes exactos cambian con la computadora, porque dependen del modelo de CPU, de la caché y de en qué núcleo corrió el programa (aquí fue `cpu_atom`).

**Claude:** además, `perf` atribuye la muestra a la instrucción donde se detiene el contador, que suele ser la siguiente a la que realmente espera (efecto *skid*). Por eso los porcentajes de instrucciones vecinas conviene leerlos como bloque y no uno a uno. Tus resultados solo son comparables con los de tus compañeros en el patrón general, no en los números.

### Distancia entre centroides durante las iteraciones

**Usuario:** ¿Cómo cambia la distancia entre centroides durante las iteraciones?

Iteración	Distancia entre centroides
Inicial	760,12
21 (mínimo)	541,59
39	aprox. 601,5
45 (final)	600,87

**Claude:** la distancia baja de 760,12 a un mínimo de 541,59 en la iteración 21, luego sube y se estabiliza cerca de 601. La rotación se recuperó bien ( $-18,00595^\circ$  contra  $-18^\circ$ ), pero la traslación no: el error en `ty` es de aproximadamente 608 (1781,98 contra 2390,35) y en `tx` de aproximadamente 37. Esto es consistente con que ICP convergió a un mínimo local.

Cambiar este chat de Cwork a chat

**Usuario:** ¿Se puede cambiar este chat de Cwork a chat?

**Claude:** según el centro de ayuda, Cwork y el chat están unificados, pero no existe una forma de convertir una conversación ya iniciada de un modo a otro. Lo que sí puedes hacer es abrir una conversación nueva y pegar el contexto que necesites.

## Comandos usados, en orden

Se respetaron los comandos de la guía para que los resultados sean comparables con los de tus compañeros. Donde no tenía el texto exacto que ejecutaste, lo indico.

#	Comando	Inciso	Para qué
1	<code>./point_cloud_collimation</code>	A	Ejecución base: métricas y centroides
2	<code>./point_cloud_collimation --export</code>	B	Ejecución con exportación de perfiles y cuadros
3	<code>perf record -g</code> <code>./point_cloud_collimation</code>	B	Muestreo con grafo de llamadas
4	<code>perf report</code> guardado en <code>perf_report.txt</code>	B	Reporte de perf (comando exacto de redirección no indicado)
5	<code>perf stat ./point_cloud_collimation</code>	B	Contadores de la ejecución normal

#	Comando	Inciso	Para qué
6	<code>perf stat ./point_cloud_collimation --export</code>	B	Contadores con exportación (costo aproximado de 9 %)
7	<code>valgrind --tool=callgrind ./point_cloud_collimation y callgrind_annotate</code>	B	Instrucciones por función (flags exactos según la guía)
8	<code>make clean &amp;&amp; make con libprofiler</code>	B	Recompilar enlazando gperftools
9	<code>CPUPROFILE=point_cloud.prof ./point_cloud_collimation</code>	B	Generar el perfil de gperftools
10	<code>google-pprof --text ./point_cloud_collimation point_cloud.prof &gt; gperftools_report.txt</code>	B	Reporte de gperftools
11	<code>g++ -std=c++17 -O2 -g -S -masm=intel ... -o point_cloud_collimation.s</code>	C	Ensamblador con sintaxis Intel
12	<code>objdump -drwC -Mintel ./point_cloud_collimation &gt; point_cloud_collimation.objdump</code>	C	Desensamblado del ejecutable
13	<code>perf annotate ... &gt; perf_annotate.txt</code>	C	Anotación de perf (comando exacto pendiente)

Pendientes de tu lado: correr `perf record` y gperftools también con `--export` para completar la sección 2.1, confirmar el comando de `perf annotate`, buscar `sin`, `cos`, `hypot` y `lround` en el desensamblado, y llenar las explicaciones de las secciones 2.1 y 3. Los incisos D (instrumentación con `std::chrono`) y E (optimización) no se han empezado.